

JammIA

Conversational chatbot on Caravaggio and Caracciolo
and the museums of Naples

Technical architecture documentation

Natural Language Processing project
Master's Degree course

Manuel Mignogna, Alessia Previdente

Contents

1	System overview	3
1.1	The flow of a turn	3
1.2	Project structure	3
2	Technology stack and libraries	4
2.1	Language model and orchestration	4
2.2	Data, voice and interface	4
2.3	Device and model selection	5
3	The ingestion pipeline	5
3.1	Extraction: the SPARQL queries	6
3.2	Transformation: <code>Extractor</code>	6
3.3	Loading: <code>Neo4jLoader</code> and the graph schema	6
4	The conversational system: LangGraph state graph	7
4.1	The state: <code>DialogState</code>	7
4.2	The graph nodes	8
4.3	Conditional routing	8
4.4	Human-in-the-loop and persistence	9
5	Request classification and decomposition	9
5.1	The output schema: <code>ModelResponse</code>	9
5.2	The three categories	10
5.3	One prompt per role	10
5.4	Prompting choices	10
5.5	Robustness	11
6	The RAG module: from natural language to Cypher	11
6.1	Chain configuration	11
6.2	Design of the Cypher prompt	12
6.3	Parallel execution and resilience	12
7	Answer synthesis	12
8	Interface and voice channel	13
8.1	Gradio interface	13
8.2	Voice: Whisper and gTTS	13

1 System overview

The project implements **JammIA**, a conversational assistant specialized in the works of the painters **Michelangelo Merisi da Caravaggio** and **Giovanni Battista Caracciolo** (known as Battistello) and the museums of Naples that display them. The user can ask questions either by voice or in writing, and the system answers with text and speech synthesis, drawing on a knowledge graph built from open sources (Wikidata and Wikipedia).

The architecture is split into **two independent subsystems**, separated in time and in responsibility:

1. **Ingestion pipeline (offline)**. Queries Wikidata via SPARQL, enriches the data with textual descriptions from Wikipedia and materializes everything into a Neo4j knowledge graph. It is run once (or whenever the dataset is to be regenerated) via `build_database.py`.
2. **Conversational system (runtime)**. On startup (`app.py`) it exposes a Gradio interface. Each dialogue turn passes through a LangGraph state graph that classifies the request, decomposes it into sub-questions, retrieves the data from the graph via Text-to-Cypher RAG and synthesizes a natural-language answer, finally converted to speech.

The founding principle is the **separation between knowledge and reasoning**: facts live in structured form in the graph, while the LLM is used as a translator (natural language $\rightarrow$ Cypher) and as a synthesizer (data $\rightarrow$ answer), not as a source of truth. This reduces hallucinations and anchors the answers to verifiable data.

1.1 The flow of a turn

A turn goes through the system in this order:

1. **Input**. The user types in the text field or speaks into the microphone; in the latter case Whisper transcribes the audio (normalized to mono float32 in the range $[-1, 1]$, with language forced to Italian).
2. **Routing**. `ChatController` forwards the question to the `DialogManager` with the session `thread_id`. If the previous turn was suspended with a clarification question, the message resumes the graph from the suspension point (`Command(resume=...)`); otherwise a new graph invocation starts.
3. **Classification**. The first node classifies the request into three outcomes: *chitchat* (the classifier itself formulates the social reply), *clarification* (the graph suspends and asks the user), *query* (the request is decomposed into self-contained atomic sub-questions, each marked `in_scope`).
4. **RAG retrieval**. Each in-scope sub-question is translated into Cypher by the LLM, run on Neo4j (in parallel across sub-questions) and produces raw rows.
5. **Synthesis**. A single LLM call fuses all the rows into a coherent Italian answer; a fixed note about the domain boundaries is appended for the out-of-scope parts.
6. **Closing**. The turn (question + answer) is appended to the history; the answer is synthesized to speech with gTTS (if the service fails, only the text remains) and returned to the UI as text + audio.

1.2 Project structure

The code is organized into Python packages by responsibility:

Package / file	Responsibility
<code>chatbot/ingestion/</code>	Building the knowledge graph: SPARQL queries, extraction, loading into Neo4j.

<code>chatbot/rag/</code>	Text-to-Cypher RAG chain over Neo4j and the query-generation / synthesis prompts.
<code>chatbot/dialog/</code>	Dialogue management: LangGraph state graph, classification, states, routing prompts.
<code>chatbot/ui/</code>	Gradio interface and session state.
<code>chatbot/speech/</code>	Speech recognition (Whisper) and speech synthesis (gTTS).
<code>chatbot/config.py</code>	Centralized configuration: credentials, models, paths, constants.
<code>query/*.psql</code>	The four parametric SPARQL queries (artists, works, museums).
<code>app.py</code> , <code>build_database.py</code>	Entry points: UI startup and database population.

2 Technology stack and libraries

The choice of libraries reflects two priorities: run everything **locally** (no dependency on paid APIs for the LLM) and rely on **established abstractions** (LangChain/LangGraph) to avoid reimplementing orchestration and graph integration.

2.1 Language model and orchestration

Library	Role and rationale
<code>langchain-ollama</code>	Local LLM (<code>ChatOllama</code>) — runs Gemma via Ollama (or compatible backends) with no cost or API keys: it is the default provider.
<code>langchain-google-genai</code>	Cloud LLM (<code>ChatGoogleGenerativeAI</code>) — alternative provider (Gemini via API key): same LangChain interface, selectable with <code>LLM_PROVIDER=gemini</code> without code changes.
<code>langgraph</code>	Dialogue state graph — models the turn as a state machine with nodes, conditional edges and human-in-the-loop (<code>interrupt</code>); includes the <code>MemorySaver</code> checkpointer.
<code>MemorySaver</code> (<code>langgraph</code>)	State persistence — in-memory checkpointer that keeps the conversation state across turns, indexed by <code>thread_id</code> ; adequate for a single-process app (state is reset on restart).
<code>langchain-neo4j</code>	RAG over the graph — provides <code>Neo4jGraph</code> and <code>GraphCypherQAChain</code> (Text-to-Cypher) ready to use.
<code>pydantic</code>	Typed models — defines the schema of the classifier output (<code>ModelResponse</code>) and the dialogue state (<code>DialogState</code>); the model's JSON output is validated with <code>model_validate_json</code> .

2.2 Data, voice and interface

Library	Role and rationale
---------	--------------------

<code>sparqlwrapper</code>	SPARQL client — queries the Wikidata endpoint; handles the JSON format and headers.
<code>neo4j</code>	Graph driver — direct access to Neo4j for the loading phase (MERGE of nodes/relationships).
<code>transformers</code> + <code>torch</code> + <code>torchaudio</code>	Speech-to-Text — run Whisper large-v3 locally for audio transcription.
<code>gtts</code>	Text-to-Speech — simple Italian speech synthesis of the final answer.
<code>gradio</code>	Web interface — chat UI with native support for microphone, audio and loading animations.
<code>mlx-lm</code>	Apple Silicon backend — enables optimized inference on Apple GPUs (the <code>-mlx</code> model variant).

2.3 Device and model selection

The configuration (`config.py`) automatically picks the available accelerator (CUDA on NVIDIA/Colab, otherwise MPS on Apple Silicon, otherwise CPU) and, accordingly, the Gemma model variant (`gemma4:e4b-mlx` on Apple, `gemma:e4b` elsewhere). All sensitive constants (Neo4j URI and credentials, model names, endpoints, API keys) are read from environment variables with default values, so they are not scattered through the code.

Chat-model creation is centralized in the `config.make_llm(temperature)` factory: `RagChain` and `DialogManager` are unaware of the provider, which is selected with `LLM_PROVIDER` (`ollama` , local default, or `gemini` with `GOOGLE_API_KEY` and a model configurable via `GEMINI_MODEL`). The rest of the pipeline (JSON parsing, state graph, RAG) is provider-agnostic, because it uses only the common LangChain chat-model interface. With the cloud provider you give up fully local execution (the questions transit through Google’s servers), in exchange for markedly better classification and synthesis quality.

3 The ingestion pipeline

The goal of this phase is to turn heterogeneous, semi-structured data (Wikidata) and discursive text (Wikipedia) into a clean, queryable graph. It is orchestrated by `pipeline.populate_database()` and consists of three stages: **extraction**, **transformation/enrichment**, **loading** (a classic ETL).

Why Wikidata and not DBpedia

As the source of the structured data, **DBpedia** was initially evaluated, a natural alternative to Wikidata and also queryable via SPARQL. In practice, however, for the works and artists in the domain DBpedia exposed properties that were **too sparse or entirely absent** (dates, dimensions, locations, relationships), insufficient to build a useful graph and to produce answers of acceptable quality. **Wikidata** was therefore chosen, whose coverage of entities and properties for this domain is clearly more complete and regular, and which offers a stable identifier (Q-ID) to anchor the nodes. The textual descriptions, instead, are drawn from the Wikipedia API (see the transformation stage), richer than the abstracts available elsewhere.

3.1 Extraction: the SPARQL queries

The four queries in `query/*.psql` query Wikidata starting from the identifiers of the two artists (`Q42207` for Caravaggio, `Q2519261` for Caracciolo). They were written with a few recurring precautions:

- **Language preference with fallback.** Labels are requested in Italian and, if absent, in English, via `COALESCE(?labelIT, ?labelEN, "default")`. This avoids empty fields when Wikidata lacks the Italian translation.
- **Aggregation to avoid duplicates.** Multi-valued properties (subjects, movements, notable works) are collected with `GROUP_CONCAT(DISTINCT ...)`; single values with `SAMPLE(...)`, so each entity produces a single row.
- **Alternative sources merged with UNION.** The location of a work is looked up both as “location” (`P276`) and as “collection” (`P195`); the museum address through several properties. This maximizes coverage of a notoriously irregular knowledge graph.

`SparqlExecutor` loads the queries, runs them with **rate-limit retry** (HTTP 429, wait and retry) and **caches the results** (`cache/sparql_cache.json`). Non-transient errors instead follow a *fail-loudly* policy: the exception propagates and stops the pipeline, so a failure is never cached as an empty result (which on the next run would mask the problem forever). The cache is crucial: without it, every database rebuild would repeat dozens of requests to Wikidata, slow and subject to throttling.

3.2 Transformation: Extractor

`Extractor` converts the raw SPARQL *bindings* into clean Python dictionaries and enriches them:

- It extracts the Q-ID from the Wikidata URIs, truncates dates to the useful part (year or ISO date), and translates coordinates from the WKT format `Point(lon lat)` into the `(lat, lon)` pair.
- For the works, it **replaces Wikidata’s terse description with the discursive introduction of the corresponding Wikipedia article**, retrieved in batches from the Italian Wikipedia API (`prop=extracts`, intro only, plain text). Here too a dedicated cache (`wikipedia_cache.json`) is used, with handling of redirects and title normalizations, and throttling to respect the service’s limits.

This choice is deliberate: Wikipedia’s descriptions are much richer and more useful for the final answer than Wikidata’s minimal strings.

3.3 Loading: Neo4jLoader and the graph schema

`Neo4jLoader` is implemented as a **context manager** (`__enter__` / `__exit__`) to guarantee the Neo4j driver is closed even in the presence of exceptions.

Nodes are inserted with `MERGE ... ON CREATE SET`, idempotent: re-running the load does not duplicate the nodes. The resulting schema is the following:

Node	Main properties
Artista	name, data_nascita, data_morte, luogo_nascita, movimenti, opere_notevoli, wikidataId
Opera	name, anno, altezza, larghezza, tecnica, soggetti, descrizione, tipo, wikidataId
Museo	name, descrizione, indirizzo, sito, telefono, fondazione, latitudine, longitudine, biglietto
Città	name

The relationships connect the entities according to the domain semantics:

```
(Opera)-[:DIPINTA_DA]->(Artista)
(Opera)-[:ESPOSTA_IN]->(Museo)
(Museo)-[:SITUATO_IN]->(Città)
```

The loading order (artists and museums before works) is not accidental: `insert_work` looks up the artist and the museum to create the relationships, so those nodes must already exist. The lookup uses `OPTIONAL MATCH` with a conditional `FOREACH` instead of a plain `MATCH`: if a reference is missing (for example a work with no known museum), the corresponding relationship is simply not created, without silently truncating the rest of the query.

At the end of the load (`create_fulltext_indexes`), two Lucene **full-text indexes** are created, `operaNameIndex` on `Opera.name` and `museoNameIndex` on `Museo.name`, with `CREATE FULLTEXT INDEX ... IF NOT EXISTS` (idempotent: `DETACH DELETE` does not remove the indexes). These indexes enable **fuzzy** title search at query time (see the RAG module).

4 The conversational system: LangGraph state graph

The heart of the runtime is `DialogManager`, which models a dialogue turn as a LangGraph **state graph** (`StateGraph`). This choice makes it possible to represent the decision flow explicitly (classify, possibly ask for clarification, retrieve and answer) with conditional edges instead of a cascade of nested `if`s.

4.1 The state: `DialogState`

The state that traverses the graph is a Pydantic model with the essential fields of the turn:

Field	Meaning
<code>question</code>	The user's current question (possibly enriched with the clarification).
<code>sub_questions</code>	List of atomic sub-questions, each with an <code>in_scope</code> flag.
<code>clarification_question</code>	The clarification question to ask the user, if needed.
<code>clarification_attempts</code>	Counter of clarification attempts (to avoid infinite loops).

<code>history</code>	The most recent turns of the conversation, as a list of <code>Turn</code> (question + answer).
<code>response</code>	The final answer (also used for chitchat).

State updates use a partial `TypedDict` (`DialogStateUpdate`): each node returns only the fields it modifies, and `LangGraph` merges them into the state. The history is kept **limited** (last 10 turns saved, last 3 passed as context to the prompts) to bound the prompt size; each `Turn` is serialized in the prompt with **explicit roles** (“Utente: ...” / “Assistente (tu): ...”), so the model recognizes that the previous replies, including offers such as “Se vuoi posso darti informazioni anche su...”, are its own, and can use them to resolve implicit references and acceptances (“sì”, “fallo”).

The lifecycle of the state is tied to the **checkpointer**: on each invocation `LangGraph` reloads the state saved for that `thread_id`, runs the nodes merging the `DialogStateUpdate`s, and at the end (or at suspension for `interrupt`) persists it again. The conversation is therefore fully reconstructible from the `thread_id` alone, and the UI does not have to carry the history on every call.

4.2 The graph nodes

Node	Function
<code>USER_PROMPT_CLASSIFICATION</code>	Classifies and decomposes the request by calling the LLM and validating the produced JSON with Pydantic.
<code>USER_INTENT_CLARIFICATION</code>	Suspends the graph (<code>interrupt</code>) and asks the user for a clarification.
<code>RESPONSE_GENERATION</code>	Runs the RAG over the in-scope sub-questions and synthesizes the answer.
<code>HISTORY_UPDATE</code>	Appends the current turn to the history and terminates.

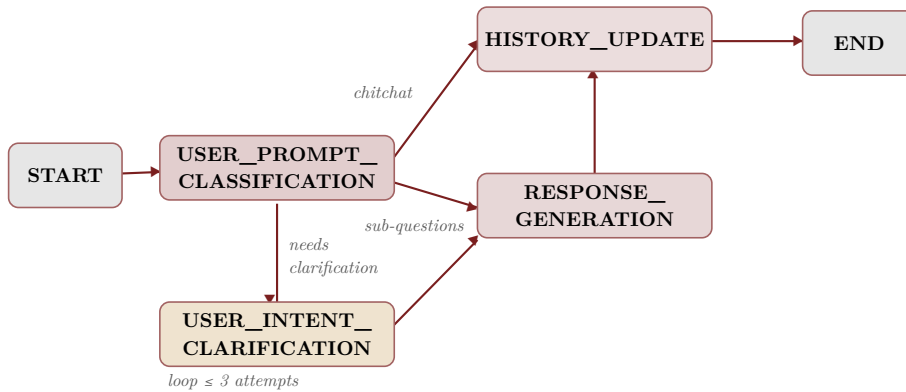


Figure 1: Nodes and edges of the dialogue state graph.

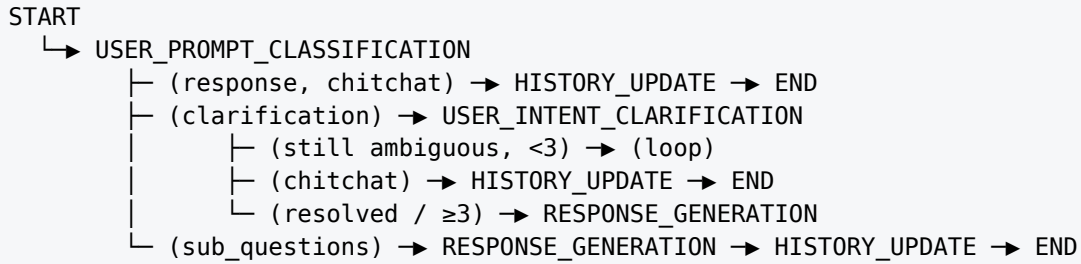
4.3 Conditional routing

After classification, a routing function (`_route_after_resolve`) decides the next node based on **which state fields are set**, with a precise priority:

1. if a `response` is present (chitchat case) → go straight to the history update, skipping the graph entirely;
2. if a `clarification_question` is present → go to the clarification node;

3. if `sub_questions` are present → go to answer generation.

The clarification node in turn has a conditional edge with three exits: if after the user's reply the request is still ambiguous and the **3 attempts** have not been exceeded, a new clarification is requested; if the user's reply turns out to be chitchat (e.g. "grazie, lascia stare"), the social reply is already ready in the state and it goes straight to the history update; otherwise it proceeds to generation. The attempt limit prevents the user from being trapped in a loop of questions: on the third failure an answer is attempted anyway with the question as it stands.



Listing 1: Flow of the dialogue state graph.

4.4 Human-in-the-loop and persistence

Clarification relies on LangGraph's `interrupt` mechanism: the graph **suspends**, returning the question to the interface, and resumes from exactly that point when the user's reply arrives (`Command(resume=...)`). For this to work across two distinct HTTP invocations a **checkpoint** is needed: the state is serialized (with `JsonPlusSerializer`, informed about the Pydantic models `Turn` and `SubQuestion`) and indexed by `thread_id`, i.e. per conversation. The checkpoint is an in-memory `MemorySaver`: sufficient for a single-process app, with the obvious trade-off that conversations are reset on restart.

The `thread_id` (UUID) is created **lazily on the first turn of each** Gradio session, not at UI construction: the initial value of `gr.State` is in fact evaluated only once at startup and copied into every session, so generating it there would make all users share the same conversation. The reset button generates a new `thread_id`, effectively wiping the history.

5 Request classification and decomposition

The first node is also the most delicate: it must decide **what** to do with the user's message before even touching the graph. It asks the LLM to produce JSON conforming to the Pydantic schema `ModelResponse`, which is then validated with `model_validate_json` (handling of malformed output is described later, in the robustness paragraph). This approach was preferred over `with_structured_output` because constrained decoding (Ollama's `format` parameter) is not guaranteed by all compatible backends.

5.1 The output schema: `ModelResponse`

```
class ModelResponse(BaseModel):
    type: Literal["query", "clarification", "chitchat"]
    sub_questions: Optional[list[SubQuestion]] = None # if type == query
    clarification_question: Optional[str] = None # if type == clarification
    response: Optional[str] = None # if type == chitchat
```

Each type uses a **single payload field**, with no ambiguous combinations: this keeps the routing logic clean (a switch on `type` decides which field to read). For chitchat the classifier fills the `response`

field directly with the already-formulated social reply, so a greeting or a thank-you is resolved in a **single call** to the LLM, without a second generation pass.

5.2 The three categories

Category	When it fires and what it produces
QUERY	The request contains at least one question about works/artists/museums. It is decomposed into atomic sub-questions, each self-contained and with an <code>in_scope</code> flag.
CLARIFICATION	The request would require a query but contains an implicit reference that neither the message nor the history resolves (e.g. “Chi l’ha dipinta?” with no work named).
CHITCHAT	Purely social messages (greetings, thanks) with no information request. The reply is produced by the classifier itself in the <code>response</code> field.

5.3 One prompt per role

The dialogue uses **three distinct prompts**, each with a single responsibility: the **classifier** (routing, query decomposition and, for chitchat, direct formulation of the social reply, with the history available to introduce itself as JammIA only on the first exchange and not repeat itself), the **Cypher prompt** (generating the query from natural language) and the **synthesis prompt** (from data to a discursive answer). Keeping them separate prevents one role’s style rules from “polluting” the others (for example, instructions about the answer’s tone have no reason to appear in the prompt that generates Cypher) and allows each to be iterated on in isolation.

5.4 Prompting choices

The classifier prompt concentrates several non-trivial instructions:

- **Reference resolution (coreference).** Each sub-question must be rewritten in fully self-contained form: demonstratives and pronouns (“questi quadri”, “lui”, “lì”) must be replaced with the explicit name of the entity, taken from the previous exchanges. Whoever reads the sub-question must not need the conversation to understand it, an essential requirement because the downstream Cypher generation receives the sub-questions in isolation.
- **Decomposition of compound requests.** A question containing more than one is split into atomic units on a single topic. This makes it possible to retrieve and then recombine the data in a controlled way.
- **Scope marking (`in_scope`) with verify-before-discarding.** Each sub-question is labeled as in- or out-of-domain. It is marked out of scope (`in_scope: false`) only when it is generic about another artist or topic **without naming any specific work or museum** (“...e quante ne ha fatte Botticelli?”): these parts do not touch the graph and get an explicit note. If instead the sub-question names a specific work or museum, it stays in scope **even if attributed to an artist not covered**, because that title must be verified in the graph: it might actually be a work of Caravaggio or Caracciolo attributed by mistake. In that case the sub-question is rewritten to look up the work by name (without constraining the wrong artist) and to ask for the true author, reporting the user’s attribution in parentheses so the synthesis can correct it.
- **Bias toward QUERY when in doubt**, so as not to reject legitimate questions, and explicit disambiguation between the two artists.
- **Accepted offers.** If the assistant’s last reply closed with a concrete offer (“Se vuoi posso darti informazioni anche su Caracciolo”) and the user accepts even generically (“sì”, “fallo”, “vai”), the

request is always QUERY: the offer is rewritten as an explicit question, applying to the offered topic the form of the user's last request.

- **Clarification as a last resort.** CLARIFICATION fires only if the reference is truly unresolvable; it is forbidden to ask for confirmation of an already-understood question (“Vuoi sapere X?” implies X is already the sub-question). This rule, together with the previous one, avoids chains of superfluous clarifications.

Consistency between prompt and schema

Since the output is constrained to the Pydantic schema, the keys shown in the prompt examples must exactly match the field names (`response` , `clarification_question` , `sub_questions`). A mismatch (for example suggesting a `text` key that does not exist in the schema) leads the model to produce a field that is discarded, leaving the expected value at `None` .

5.5 Robustness

With small local models the JSON output can occasionally be malformed or wrapped in spurious text. The defense is two-layered: the **tolerant extraction** (`_extract_json`) isolates the portion between the first `{` and the last `}` , recovering the most common cases (markdown fences, prefixes like `json` , prepended text); for unrecoverable cases a **defensive fallback** intercepts the error and degrades to a clarification request, instead of crashing the whole turn. The moderate `temperature` (0.2) keeps the output reasonably stable without freezing it completely. Likewise, if the graph should end without a response, the UI receives a courtesy message instead of an exception.

6 The RAG module: from natural language to Cypher

Information retrieval follows the **Text-to-Cypher RAG** paradigm: instead of searching a vector index, the LLM translates the question into a Cypher query that is run on the Neo4j graph. `RagChain` encapsulates the graph connection, the LLM and LangChain's `GraphCypherQChain` .

6.1 Chain configuration

```
GraphCypherQChain.from_llm(
    llm=self.llm,
    graph=Neo4jGraph(...),
    cypher_prompt=CYPHER_PROMPT,
    return_direct=True,      # returns the raw rows, skips the internal synthesis
    validate_cypher=True,    # fixes relationship directions according to the schema
    allow_dangerous_requests=True,
)
```

Two choices deserve an explanation:

- **return_direct=True** . Normally `GraphCypherQChain` makes **two** LLM calls: one to generate the Cypher, one to turn the results into prose. Here the second is disabled: the chain is used **only for retrieval** and returns the raw rows. Turning them into a discursive answer is centralized elsewhere, in a **single** synthesis call for the whole turn. This way a question made of three sub-questions costs one synthesis instead of three, saving calls.
- **validate_cypher=True** . Automatically fixes the direction of the arrows in the relationships to match the schema, preventing a backwards-oriented edge from silently returning an empty result.

The **graph schema is automatically injected** into the prompt by `Neo4jGraph`, which introspects it: the model therefore knows the available labels, properties and relationships when it generates the query.

6.2 Design of the Cypher prompt

The generation prompt (`CYPHER_GENERATION_TEMPLATE`) encodes a set of rules born from the typical mistakes of small models on Text-to-Cypher:

- **Bidirectional relationships without arrows.** Flat syntax `-[:REL]-` without `</>` is enforced. A model guessing the wrong direction would produce empty results; removing the orientation makes the match work regardless.
- **Case-insensitive.** Always compare with `toLower(...)` on both sides, so as not to fail on case differences.
- **Fuzzy search of works and museums.** Titles are searched neither by equality nor with `CONTAINS` on the whole phrase, but through the Lucene **full-text** indexes with the fuzzy operator `~`. The search string is built by discarding articles and prepositions (“di”/“della”/“il”...) and making each `CONTENT` word both **required** (prefix `+`) and typo-tolerant (suffix `~`): `CALL db.index.fulltext.queryNodes("operaNameIndex", "+sette~ +opere~. This +misericordia~") YIELD node, score ... ORDER BY score DESC LIMIT 1`. This solves two opposite needs in one shot: a different preposition (“della” instead of “di”) or a typo (`Flagellazzione`) still matches thanks to the Levenshtein distance, whereas a title with an invented subject word (e.g. `Flagellazione di Babbo Natale`) makes a non-existent word required and therefore returns **zero rows**, cleanly falling into the “work not found” case instead of being passed off as a real work. The true author (`a.name`) is always returned, to verify and correct any wrong attributions.
- **Mandatory aliases on aggregates.** Every `count` / `collect` / `sum` must have a readable `AS`, so the returned rows have keys understandable by the synthesis.
- **Handling multiple entities.** When the question concerns several works, a single pattern with `WHERE ... IN [...]` is enforced, returning each entity with its own value, rather than binding them all to the same node (a mistake that often produces empty results).

6.3 Parallel execution and resilience

In the generation node, the **in-scope** sub-questions are deduplicated and queried **in parallel** with a `ThreadPoolExecutor` (up to 5 workers): since each query is independent and spends most of its time waiting on network/DB, parallelism cuts the turn’s latency. Each query also has a **retry policy** (up to 3 attempts) **on exceptions only** (invalid Cypher, connection errors): an empty result on a successful query is a legitimate answer (“nothing there”) and is returned immediately, without wasting LLM calls on pointless attempts.

7 Answer synthesis

Once the rows are retrieved from the graph, a **single** LLM call (`COMBINE_PROMPTS_TEMPLATE`) fuses them into a coherent Italian answer. The synthesis prompt is heavily constrained in style, to counter the undesirable tendencies of generative models:

- **concise** answer (3–4 sentences), straight to the data, with no preambles or closing invitations;
- **no citing the source** (“secondo il database”, “nel mio archivio”): the model must answer as if it already knew the facts;
- **no disclaimers** when the data is present, and use of **all and only** the values present (if it is a list, list them all without inventing any);

- if the data for a sub-question is empty, that information is declared unavailable without inventing, while still answering the others;
- a cordial, direct tone, in clear Italian, also suitable for speech synthesis;
- **exact title and corrections**: the work must always be named with the exact title returned by the data (`o.name`), never with the form typed by the user; if the latter differed (different preposition, typo), the answer points it out politely, indicating the correct form. Likewise, if the data shows an author different from the one the user attributed, the answer corrects the attribution;
- **subject verification (safety net)**: invented titles are already filtered out upstream by the query (required content words, see the RAG module), but as a further defense the synthesis still compares the requested subject word with the title in the data: if they differ (e.g. “Babbo Natale” vs “Cristo”) it does not pass the result off as the requested work and declares that the requested one does not exist, possibly citing the similar one that actually exists;
- **constrained closing suggestion**: the answer may close with an offer (“Se vuoi posso darti informazioni anche su...”), but only about in-scope entities, explicitly named and not yet covered. The concreteness constraint is not cosmetic: an explicit offer is what allows the classifier, on the next turn, to resolve a generic acceptance (“sì”, “fallo”) without asking for clarification.

For the **out-of-scope** sub-questions a fixed note is reserved that recalls the domain boundaries, concatenated to the in-scope answer. This way a mixed request receives a complete, honest answer: real data where the graph has it, an explicit boundary where it does not.

Why synthesis is separate from retrieval

Separating retrieval (one query per sub-question) from synthesis (a single call per turn) is the architectural choice that keeps the number of LLM invocations low and centralizes style control in a single prompt, instead of scattering it across the chain’s internal generation.

8 Interface and voice channel

8.1 Gradio interface

`ChatController` builds the interface, branded JammIA, with a Gradio theme in Naples-blue tones (passed to `launch()`, as required by Gradio 6), and exposes its handlers. Each turn is split into **two chained events** (`step 1` .then(`step 2`)): the first immediately adds the user’s message to the chat (instant feedback), the second computes the answer. Two inputs are supported, text and microphone, which converge on the same generation step.

The session state (`SessionState`, in `gr.State`) keeps two pieces of information: the conversation’s `thread_id` (created lazily on the first turn, see above) and an `awaiting_clarification` flag, indicating whether the next message should be routed as a **reply to a clarification** rather than as a new question. The reset button regenerates the `thread_id`, effectively wiping the history.

8.2 Voice: Whisper and gTTS

Speech recognition (`SpeechToText`) uses **Whisper large-v3** through the `transformers` pipeline, forcing the Italian language. The (`sample_rate`, `ndarray`) audio produced by Gradio’s microphone arrives as integers (typically `int16`, possibly stereo) and is **normalized** into the format Whisper expects (mono, `float32`, values in `[-1,1]`); without this normalization transcription degrades significantly. Speech synthesis (`TextToSpeech`) uses **gTTS** and writes each answer to a **separate temporary file** (no shared file: this avoids race conditions between concurrent sessions and stale audio cached by the browser). Synthesis is **fault-tolerant**: if the TTS service fails, the turn still shows the text answer, simply without audio.